

LANGGRAPH — PAGE 1: CORE ARCHITECTURE

Purpose

LangGraph is a low-level orchestration framework for long-running, stateful agent workflows. A graph makes control flow explicit: branches, cycles, parallel fan-out, retries, and human approval.

Why not only a chain?

Chains are mostly linear. Agents need loops:
reason → tool → observe → reason.

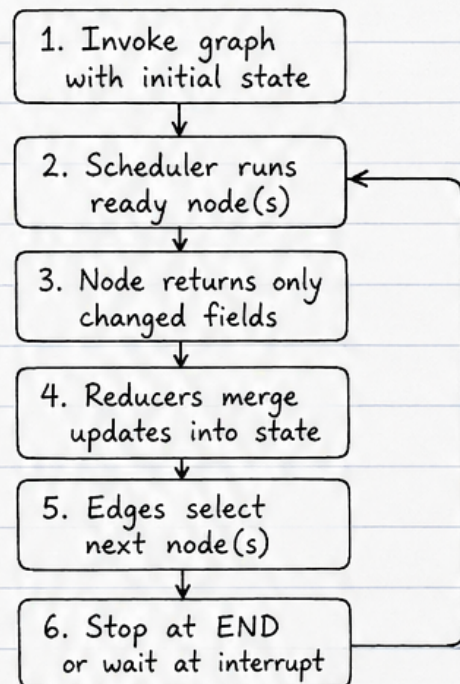
Graphs make these loops observable, resumable and testable.

Main abstractions

- **StateGraph**: graph whose nodes read/write shared state
- **State**: typed schema for workflow data
- **Node**: sync/async function: State → partial state update
- **Edge**: transition between nodes
- **START / END**: special entry and terminal nodes
- **Reducer**: rule that merges updates to the same state key

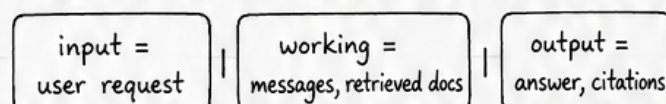
Execution model

1. Invoke graph with initial state
2. Scheduler runs ready node(s)
3. Node returns only changed fields
4. Reducers merge updates into state
5. Edges select next node(s)
6. Stop at **END** or wait at interrupt



State design

Keep state minimal and explicit. Separate:



State is application data, not automatically model memory.

Key idea

Graph = state + nodes + edges + persistence.
The graph coordinates; nodes contain business logic.

Interview line

Use LangGraph when I need explicit, durable control of an agent loop — not just prompt chaining.

LANGGRAPH — PAGE 2: STATE, NODES & ROUTING

Typed state

Use TypedDict, Pydantic, or dataclass-style schemas. Nodes return partial updates, never mutate shared state in place.

Example

```
class AgentState(TypedDict):  
    messages: Annotated[list, add_messages]  
    query: str  
    docs: list[str]  
    answer: str
```

Reducers

A reducer defines merge behavior when a key receives an update. `add_messages` appends/updates messages by id. Default behavior replaces the old value.

Use reducers for append-only logs, accumulated documents, counters or parallel writes. Avoid ambiguous reducers.

Nodes

Good node = single responsibility, deterministic I/O contract, structured output.

Examples: `retrieve_docs`, `grade_docs`, `call_model`, `execute_tool`, `format_answer`.

Make external calls async where possible. Validate LLM JSON before putting it in state.

Routing options

- `add_edge(A, B)`: fixed transition
- `add_conditional_edges(A, router)`: route based on state
- `Command`: update state + choose goto in one return
- `Send`: fan out dynamic work to many worker nodes

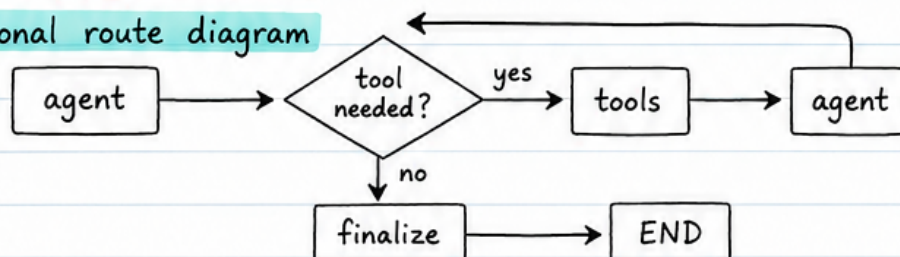
Router pattern

```
if state['needs_tool']:  
    return 'tools'  
if state['retries'] >= 3:  
    return 'fallback'  
return 'respond'
```

Command pattern

```
return Command(update={'answer': result}, goto='finalize')
```

Conditional route diagram



Design rule

Put decisions in routers; put actions in nodes. This keeps branching auditable.

LANGGRAPH — PAGE 3: MEMORY, PERSISTENCE & HITL

Checkpointing

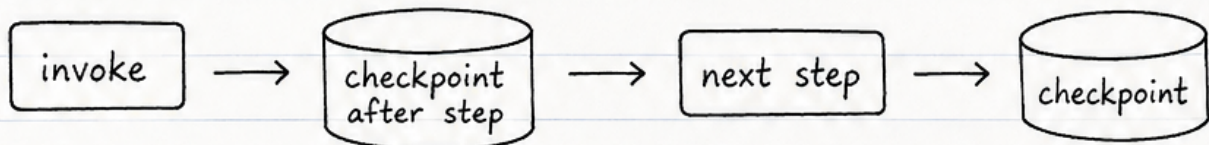
A checkpointer saves graph state at execution steps. It enables memory, recovery, replay and human-in-the-loop.

Thread ID

Pass `configurable={'thread_id': 'user-123'}`.

- Same `thread_id` = continue the same conversation/workflow.
- Different `thread_id` = isolated workflow history.

Checkpoint lifecycle



On failure, resume from saved state instead of rerunning everything.

Checkpointing choices

- MemorySaver: local development/testing
- SQLite/Postgres: durable application storage
- Production: use durable storage, backup policy, retention and access control.

Interrupts / human-in-the-loop

- Call `interrupt(payload)` inside a node when approval is required.
- Graph pauses and returns an interrupt event.
- Reviewer approves/edits/rejects, then resume with `Command(resume=value)`.

Important

- On resume, the interrupted node may rerun. Code before interrupt must be idempotent; never repeat a payment, email or destructive tool call.

Use cases

- approve high-risk action
- correct extracted data
- review generated SQL
- request missing information

Memory types

- Short-term: state + checkpoints inside one thread
- Long-term: store user facts separately; retrieve only relevant facts
- Do not put unlimited chat history into prompts—summarize or trim.

Interview line

Persistence turns an agent run into a recoverable workflow, while interrupt adds controlled human judgment.

Tool calling

Bind tools to the model; model selects a tool; ToolNode executes validated calls; tool result returns as a message; agent reasons again. Never let raw model text run an unsafe action. Use typed tool args, authorization, allowlists and confirmation for risky operations.

Streaming

`stream` / `astream` exposes progress while the graph runs.

Common modes:

- `values`: full state after each step
- `updates`: only state delta per node
- `messages`: token-level model output
- `custom`: application progress events

Use streaming for chat UX, progress screens and observability.

Retries & errors

Classify errors: transient (timeout, rate limit), deterministic (bad input), business failure (no data).

Retry only transient errors with exponential backoff and a maximum attempt count.

Route failures to fallback / human review; store error type and correlation id in state.

Parallelism

Send creates map-style fan-out. Use a reducer to combine worker results.

Limit concurrency; parallel tool calls can hit rate limits and reorder outputs.

Subgraphs

Use a subgraph for reusable bounded workflows: RAG retrieval, document extraction, approval flow. Keep clear parent-child state mapping.

Testing & observability

Unit-test nodes and routers with fixed state. Test graph paths, interrupts, retries and resume.

Trace node name, latency, tokens, tool input/output, errors and state transitions. Never log secrets or sensitive prompts.

Production checklist

`schemas` | `idempotency` | `timeouts` | `retries` | `checkpointing` | `auth` |
`tracing` | `evaluation` | `cost limits`

Strong interview answer

LangGraph is the orchestration layer: I define typed state, isolated nodes, explicit routing, durable checkpoints and guarded tool execution so agent behavior is controllable in production.

LANGGRAPH — PAGE 5: BUILDING A GRAPH

Minimal build sequence

1. Define state schema
2. Create graph = StateGraph(State)
3. graph.add_node('name', function)
4. graph.add_edge(START, 'first')
5. graph.add_conditional_edges('agent', router)
6. graph.add_edge('final', END)
7. app = graph.compile(checkpointer=checkpointer)
8. app.invoke(input, config)

Compile meaning

Compilation validates and turns the graph definition into an executable Runnable. Attach checkpointer, interrupt settings, cache or debug options here.

Invocation API

- invoke: run to completion and return final state
- ainvoke: async version
- stream / astream: yield state or token events
- get_state: inspect current checkpointed state
- get_state_history: inspect prior checkpoints
- update_state: manually patch state when needed

Config

```
config = {'configurable': {'thread_id': 't-42'},
          'recursion_limit': 25,
          'tags': ['prod', 'rag']}
```

Recursion limit

Protects against accidental infinite loops. Set a sensible limit and make an explicit retry/exit condition.

Message handling

Use add_messages for chat state. Preserve roles and tool-call IDs; do not flatten messages into strings.

Structured model output

Use a schema for route decisions and extracted data. Validate before state update; if invalid, retry once or route to repair.

Cache & side effects

Cache pure, expensive steps such as retrieval. Do not blindly cache user-specific or time-sensitive results.

Keep side effects behind a dedicated node with idempotency key.

Debugging

Visualize graph, use debug/stream events, inspect state history, and reproduce with the same input + thread.

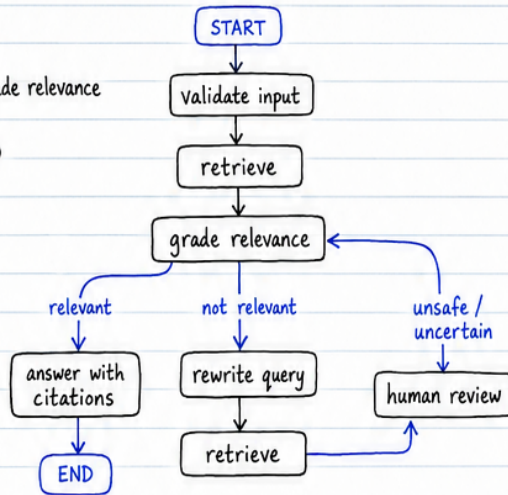
① Production RAG agent flow

START → validate input → retrieve → grade relevance

relevant → answer with citations → END

not relevant → rewrite query → retrieve

unsafe / uncertain → human review



② Agent state example

request_id, user_id, messages, query, retrieved_docs,
tool_results, retry_count, decision, answer, citations, error

③ Separation of concerns

Graph: orchestration and transitions

Nodes: model, retrieval, tool, business logic

Service layer: auth, DB, external APIs

Storage: checkpoints + vector DB + long-term memory

API layer: request validation, streaming, rate limit

④ Choosing single vs multi-agent

Start with one graph and specialized nodes.

Use multi-agent only when roles have separate tools, context or ownership. More agents add routing cost, latency and failure paths.

⑤ Evaluation

Measure task success, groundedness, tool correctness, latency, token cost, retry rate, interrupt rate and path distribution. Create fixed test cases for happy path, no-result, malformed tool args, timeout, unsafe request and resume after interrupt.

⑥ Security

Validate all tool inputs server-side. Apply least privilege, tenant isolation, PII redaction, audit log and output guardrails.

Never expose system prompts, secrets, raw database access or unrestricted shell tools.

⑦ Common mistakes

- giant untyped state
- business decisions buried inside prompts
- no terminal condition
- non-idempotent action before interrupt
- retrying validation errors forever
- treating checkpoint history as permanent user memory

⑧ 30-second answer

I use LangGraph to model agents as explicit state machines. I keep typed state, narrow nodes, deterministic routers, durable checkpoints and safe tools. That lets me inspect paths, resume failures, add approval, evaluate quality and operate the workflow reliably.